

EXERCICIO FINAL

Programação de Sistemas Distribuídos

*Desenvolvimento de um Sistema ERP
Distribuído com FastAPI*

26 alunos • 6 grupos • 4 módulos + Core + Frontend Portal

Nível de complexidade: **Intermediário**

Prazo final de entrega: 01 de junho de 2025

FastAPI • REST APIs • SQLAlchemy • JWT • Frontend Livre • pytest

Docente: Gleydes Oliveira

Contents

1	Visão Geral do Exercício	3
1.1	Objetivos de Aprendizagem	3
1.2	Tecnologias por Camada	3
2	Formação dos Grupos — Sorteio	4
2.1	Procedimento do Sorteio	4
2.2	Composição dos Grupos	4
3	Arquitetura do Sistema	5
3.1	Diagrama de Comunicação	5
3.2	Responsabilidades do Core (G0)	5
3.3	Responsabilidades do Frontend Portal (G1)	5
3.4	Regras de Integração	6
4	Especificação dos Módulos	7
4.1	Core — API Gateway e Autenticação (G0)	7
4.2	Frontend Portal — ERP Web (G1)	7
4.3	Módulo Financeiro (G2)	9
4.4	Módulo RH / Folha de Pagamento (G3)	9
4.5	Módulo Estoque / Almoxarifado (G4)	10
4.6	Módulo Compras / Fornecedores (G5)	11
5	Requisitos Técnicos Comuns	13
5.1	Estrutura de Projeto (Backend)	13
5.2	Estrutura de Projeto (Frontend)	13
5.3	Padrões de API (Backend)	13
5.4	Segurança	13
5.5	Testes (Backend)	13
5.6	Estrutura de Diretórios Sugerida	13
6	Entregáveis	15
7	Cronograma Sugerido	16
8	CrITÉRIOS de Avaliação	17
8.1	Penalizações	17
8.2	Bônus (até +1,0)	17
9	Template de Documentação de Requisitos	18
9.1	Documento de Visão do Módulo	18
9.2	Diagrama de Entidades (Backend) / Wireframes (Frontend)	18
9.3	Especificação OpenAPI	18
9.4	Documento de Integração (módulos G2–G5)	18
9.5	Documento de Integração (Frontend — G1)	18
10	Dicas e Referências	19
10.1	Boas Práticas	19

10.2 Referências	19
----------------------------	----

1. Visão Geral do Exercício

Este exercício final simula o desenvolvimento de um **Sistema ERP (Enterprise Resource Planning) distribuído**, onde cada grupo é responsável por um componente independente que se integra ao conjunto por meio de APIs REST construídas com **FastAPI** e um **Portal Web (Frontend)** que consome todos os módulos.

O prazo final e improrrogável para todas as entregas é **01 de junho de 2025**.

1.1 Objetivos de Aprendizagem

- Projetar e implementar APIs REST com FastAPI em nível intermediário;
- Aplicar conceitos de arquitetura de microsserviços e integração entre sistemas;
- Realizar levantamento de requisitos e modelagem de dados para um domínio real;
- Documentar APIs com OpenAPI/Swagger e escrever testes automatizados;
- Desenvolver um portal web integrado consumindo múltiplas APIs via JWT;
- Trabalhar em equipe com controle de versão (Git) e revisão de código.

1.2 Tecnologias por Camada

Camada	Tecnologias Obrigatórias	Observação
Backend (todos)	Python 3.11+ · FastAPI · Uvicorn · SQLAlchemy · Alembic	
Banco de dados	PostgreSQL (prod) ou SQLite (dev)	1 banco por serviço
Autenticação	JWT via Core — python-jose ou PyJWT	Centralizado no Core
Documentação	Swagger UI (/docs) · ReDoc (/redoc) · openapi.json	Gerado pelo FastAPI
Testes	pytest + httpx (TestClient) · pytest-cov	Min. 70% cobertura
Frontend	Livre escolha (React, Vue, Angular, Svelte...)	Deve consumir o Core
Versionamento	Git + GitHub — 1 repositório por grupo	Commits convencionais

Table 1: Stack tecnológica do projeto

2. Formação dos Grupos — Sorteio

A divisão dos alunos pelos grupos será realizada por **sorteio público** utilizando o site `sorteador.com.br`, garantindo imparcialidade na distribuição.

2.1 Procedimento do Sorteio

1. O professor acessa **sorteador.com.br** em sala, com projetor visível a todos;
2. Os 26 nomes são inseridos na lista e embaralhados;
3. Os primeiros **2 nomes** sorteados formam o **Grupo Core (G0)**;
4. Os próximos **4 nomes** formam o **Grupo Frontend (G1)**;
5. Os blocos seguintes de **4 nomes** formam os grupos G2, G3, G4 e G5 (módulos backend);
6. O resultado é registrado em ata e publicado no AVA.

Importante: Não haverá troca de grupo após o sorteio, salvo em casos justificados e aprovados pelo professor com antecedência mínima de 48 h.

2.2 Composição dos Grupos

G	Módulo	Integrantes	Responsabilidade Principal	Porta
G0	Core / API Gateway	2	Auth JWT, roteamento, service registry, health-check	:8000
G1	Frontend Portal	4	Portal web integrado a todos os módulos	:3000
G2	Financeiro	4	Contas a pagar/receber, fluxo de caixa	:8001
G3	RH / Folha de Pagamento	4	Funcionários, cargos, benefícios, holerite	:8002
G4	Estoque / Almoxiarifado	4	Produtos, movimentações, alertas	:8003
G5	Compras / Fornecedores	4	Fornecedores, cotações, ordens de compra	:8004

Os 8 alunos restantes (pos. 19–26) são distribuídos pelo sorteio: +2 por grupo (G2 a G5).

Table 2: Composição dos grupos após sorteio

3. Arquitetura do Sistema

O sistema segue uma arquitetura de **microsserviços simplificada**. O **Core** centraliza autenticação e registro de serviços; os **módulos backend** são APIs independentes; o **Frontend Portal** é o único cliente humano, consumindo todos os módulos via JWT emitido pelo Core.

3.1 Diagrama de Comunicação

```
[Navegador / Usuario]
|
v
[Frontend Portal :3000] <-- React / Vue / Angular / ...
|
+-- POST /auth/login -----> [Core :8000]
|   <token JWT>
|
+-- GET /financeiro/... -----> [Modulo Financeiro :8001]
|
+-- GET /rh/... -----> [Modulo RH/Folha :8002]
|
+-- GET /estoque/... -----> [Modulo Estoque :8003]
|
+-- GET /compras/... -----> [Modulo Compras :8004]

Comunicacao interna (backend-to-backend):
Compras :8004 --> POST /estoque/movimentacoes/entrada --> :8003
Compras :8004 --> POST /financeiro/contas-pagar --> :8001
```

3.2 Responsabilidades do Core (G0)

- Cadastro, login e refresh de token (JWT — access 60 min, refresh 7 dias);
- Endpoint `/auth/verify` para validação de token pelos módulos;
- *Service registry* básico: tabela de módulos registrados e suas URLs;
- Health-check agregado (GET `/health`) que consulta todos os módulos;
- Documentação do contrato de autenticação para uso pelo Frontend e pelos módulos.

3.3 Responsabilidades do Frontend Portal (G1)

- Tela de login integrada ao Core (obter e armazenar JWT);
- Dashboard principal com **indicadores reais** de cada módulo (cards com totais, alertas, resumos);
- Telas de CRUD para as principais entidades de cada módulo;
- Controle de sessão: renovação de token e logout;
- Tratamento de erros de API (401, 403, 404, 500) com feedback ao usuário.

3.4 Regras de Integração

- Todo endpoint protegido valida JWT via Core (`/auth/verify`) ou chave compartilhada;
- Módulos backend **não compartilham banco** — comunicação apenas via API;
- O Frontend **nunca** chama o banco diretamente — somente via APIs REST;
- Cada serviço expõe `GET /health` retornando `200 OK` com status;
- Chamadas cross-módulo (Compras → Estoque/Financeiro) tratam falhas com roll-back lógico.

4. Especificação dos Módulos

4.1 Core — API Gateway e Autenticação (G0)

Ponto de entrada do sistema. Gerencia identidades e serve como referência de autenticação para todos os outros serviços.

Requisitos Funcionais

- **RF01** — Cadastro de usuário com nome, e-mail, senha (hash bcrypt) e perfil (admin/user);
- **RF02** — Login retornando `access_token` (JWT, exp. 60 min) e `refresh_token` (exp. 7 dias);
- **RF03** — Renovação de token via refresh;
- **RF04** — Endpoint de verificação de token para uso interno dos módulos;
- **RF05** — Listagem e ativação/desativação de usuários (somente admin);
- **RF06** — Registro de módulos (URL + porta) e health-check agregado.

Requisitos Não-Funcionais

- **RNF01** — Senha nunca trafega em texto puro; armazenada com bcrypt;
- **RNF02** — Token JWT assinado com chave secreta via variável de ambiente;
- **RNF03** — CORS configurado para aceitar requisições do Frontend (:3000).

Endpoints Mínimos

Método	Endpoint	Descrição
POST	/auth/register	Cadastrar novo usuário
POST	/auth/login	Autenticar e receber tokens JWT
POST	/auth/refresh	Renovar access_token
POST	/auth/verify	Validar token (uso interno)
GET	/users	Listar usuários (admin)
PATCH	/users/{id}/status	Ativar ou desativar usuário
GET	/health	Status agregado de todos os módulos

4.2 Frontend Portal — ERP Web (G1)

Portal ERP — Grupo 1

O Frontend é o único ponto de interação humana com o sistema. Deve ser desenvolvido com **tecnologia de livre escolha** (React, Vue, Angular, Svelte, etc.), consumir os módulos via API REST e autenticar-se via JWT emitido pelo Core.

Requisitos Funcionais

- **RF01** — Tela de login com e-mail e senha integrada ao Core (/auth/login);
- **RF02** — Dashboard principal com **cards de indicadores reais**:
 - Financeiro: total de contas vencidas e saldo do fluxo de caixa do mês;
 - RH: total de funcionários ativos e folhas abertas;
 - Estoque: número de produtos em alerta de reposição;
 - Compras: ordens de compra em aberto.
- **RF03** — Módulo Financeiro: listagem e criação de contas a pagar e a receber;
- **RF04** — Módulo RH: listagem de funcionários e visualização de holerite;
- **RF05** — Módulo Estoque: listagem de produtos, saldo e alertas;
- **RF06** — Módulo Compras: listagem de fornecedores e ordens de compra;
- **RF07** — Logout com remoção do token e redirecionamento ao login;
- **RF08** — Feedback visual para erros de API (toast/alerta com mensagem).

Requisitos Não-Funcionais

- **RNF01** — Token JWT armazenado de forma segura (preferencialmente `httpOnly` cookie OU `sessionStorage`; nunca `localStorage` em produção);
- **RNF02** — Renovação automática do token antes da expiração;
- **RNF03** — Interface responsiva (desktop e tablet);
- **RNF04** — Rotas protegidas: redirecionamento ao login se o token for inválido ou ausente;
- **RNF05** — README com instruções de instalação, variáveis de ambiente e como rodar.

Sugestões de Tecnologia (livre escolha)

Framework	Pontos Fortes	Indicado para
React + Vite	Ecossistema vasto, componentes reutilizáveis	<i>Projetos com muita interatividade</i>
Vue 3 + Vite	Curva de aprendizado suave, ótima documentação	<i>Iniciantes em frontend</i>
Angular	Estrutura completa e opinada, TypeScript nativo	<i>Quem já conhece o framework</i>
Svelte / SvelteKit	Bundle mínimo, sintaxe simples	<i>Prototipagem rápida</i>

4.3 Módulo Financeiro (G2)

Requisitos Funcionais

- **RF01** — CRUD de contas a pagar (fornecedor, valor, vencimento, status);
- **RF02** — CRUD de contas a receber (cliente, valor, vencimento, status);
- **RF03** — Registrar pagamento/recebimento (status → “liquidado”, data de liquidação);
- **RF04** — Relatório de fluxo de caixa por período (entradas – saídas = saldo);
- **RF05** — Receber notificação do módulo Compras para criar conta a pagar automaticamente;
- **RF06** — Listar contas vencidas (vencimento < hoje e status ≠ liquidado).

Requisitos Não-Funcionais

- **RNF01** — Valores monetários como `NUMERIC(15,2)`; nunca `float`;
- **RNF02** — Autenticação JWT obrigatória em todos os endpoints;
- **RNF03** — Operações de pagamento idempotentes (prevenir duplo pagamento).

Endpoints Mínimos

Método	Endpoint	Descrição
GET	/financeiro/contas-pagar	Listar c/ filtros e paginação
POST	/financeiro/contas-pagar	Criar conta a pagar
PUT	/financeiro/contas-pagar/{id}	Editar conta a pagar
PATCH	/financeiro/contas-pagar/{id}	Registrar pagamento
GET	/financeiro/contas-receber	Listar c/ filtros e paginação
POST	/financeiro/contas-receber	Criar conta a receber
PATCH	/financeiro/contas-receber/{id}	Registrar recebimento
GET	/financeiro/fluxo-caixa	Relatório de fluxo de caixa
GET	/financeiro/contas-vencidas	Contas vencidas e pendentes

Integração com outros módulos

Compras → **Financeiro**: ao aprovar recebimento de OC, Compras envia `POST /financeiro/contas-pagar` com os dados da fatura.

Frontend → **Financeiro**: exibe saldo do fluxo de caixa e total de vencidas no dashboard.

4.4 Módulo RH / Folha de Pagamento (G3)

Requisitos Funcionais

- **RF01** — CRUD de funcionários (nome, CPF, cargo, departamento, salário base, admissão);
- **RF02** — CRUD de cargos com faixa salarial mínima e máxima;
- **RF03** — CRUD de benefícios (VR, plano de saúde, VT) e associação a funcionários;
- **RF04** — Calcular folha: salário + benefícios – descontos (INSS e IRRF simplificados);
- **RF05** — Gerar holerite individual por funcionário e mês;
- **RF06** — Histórico de alterações salariais.

Requisitos Não-Funcionais

- **RNF01** — CPF mascarado nas respostas; nunca exposto completo em listagens;
- **RNF02** — Folha fechada é imutável (não pode ser recalculada após fechamento);
- **RNF03** — Autenticação JWT obrigatória.

Endpoints Mínimos

Método	Endpoint	Descrição
GET	/rh/funcionarios	Listar c/ paginação e filtros
POST	/rh/funcionarios	Admitir funcionário
GET	/rh/funcionarios/{id}	Detalhar funcionário
PUT	/rh/funcionarios/{id}	Atualizar dados
DELETE	/rh/funcionarios/{id}	Desativar (soft delete)
GET	/rh/cargos	Listar cargos
POST	/rh/cargos	Criar cargo
POST	/rh/folha/{ano}/{mes}	Calcular folha do mês
GET	/rh/folha/{ano}/{mes}	Listar folha calculada
GET	/rh/holerite/{func_id}/{ano}	Holerite individual

Integração com outros módulos

Frontend → **RH**: exibe total de funcionários ativos e folhas abertas no dashboard.

4.5 Módulo Estoque / Almoxarifado (G4)

Requisitos Funcionais

- **RF01** — CRUD de produtos (código, descrição, unidade, qtd., estoque mínimo, custo);

- **RF02** — Registrar entrada (quantidade, fornecedor, nota fiscal);
- **RF03** — Registrar saída (quantidade, motivo, centro de custo);
- **RF04** — Consultar posição de estoque em tempo real;
- **RF05** — Listar produtos abaixo do estoque mínimo (alerta de reposição);
- **RF06** — Histórico de movimentações por produto e período;
- **RF07** — Inventário: ajuste de saldo com justificativa.

Requisitos Não-Funcionais

- **RNF01** — Quantidade em estoque nunca negativa (validação na camada de serviço);
- **RNF02** — Audit trail: usuário, data e hora em toda movimentação;
- **RNF03** — Autenticação JWT obrigatória.

Endpoints Mínimos

Método	Endpoint	Descrição
GET	/estoque/produtos	Listar c/ filtros e paginação
POST	/estoque/produtos	Cadastrar produto
GET	/estoque/produtos/{id}	Detalhar produto e saldo
PUT	/estoque/produtos/{id}	Atualizar produto
POST	/estoque/movimentacoes/entrada	Registrar entrada
POST	/estoque/movimentacoes/saida	Registrar saída
GET	/estoque/movimentacoes/{prod_id}	Histórico
GET	/estoque/alertas	Produtos abaixo do mínimo
POST	/estoque/inventario/{prod_id}	Ajuste com justificativa

Integração com outros módulos

Compras → **Estoque:** POST /estoque/movimentacoes/entrada ao receber mercadoria.

Frontend → **Estoque:** exibe count de produtos em alerta no dashboard.

4.6 Módulo Compras / Fornecedores (G5)

Requisitos Funcionais

- **RF01** — CRUD de fornecedores (CNPJ, razão social, contato, prazo de entrega);
- **RF02** — Criar solicitação de compra (produto, quantidade, justificativa);
- **RF03** — Criar cotação associada a solicitação (fornecedor + preço);

- **RF04** — Aprovar cotação e gerar Ordem de Compra com número sequencial;
- **RF05** — Registrar recebimento parcial ou total da OC;
- **RF06** — Ao receber: notificar Estoque (entrada) e Financeiro (conta a pagar);
- **RF07** — Listar OCs por status: aberta, parcial, encerrada.

Requisitos Não-Funcionais

- **RNF01** — Histórico de mudança de status em toda OC;
- **RNF02** — Rollback lógico se Estoque ou Financeiro retornarem erro;
- **RNF03** — Autenticação JWT obrigatória.

Endpoints Mínimos

Método	Endpoint	Descrição
GET	/compras/fornecedores	Listar fornecedores
POST	/compras/fornecedores	Cadastrar fornecedor
PUT	/compras/fornecedores/{id}	Atualizar fornecedor
GET	/compras/solicitacoes	Listar solicitações
POST	/compras/solicitacoes	Criar solicitação
POST	/compras/cotacoes	Registrar cotação
POST	/compras/cotacoes/{id}/aprovar	Aprovar e gerar OC
GET	/compras/ordens	Listar Ordens de Compra
GET	/compras/ordens/{id}	Detalhar OC
POST	/compras/ordens/{id}/receber	Receber (dispara integrações)

Integração com outros módulos

Compras → **Estoque**: POST /estoque/movimentacoes/entrada.

Compras → **Financeiro**: POST /financeiro/contas-pagar.

Frontend → **Compras**: exibe OCs em aberto no dashboard.

5. Requisitos Técnicos Comuns

5.1 Estrutura de Projeto (Backend)

- Diretórios: `app/core/`, `app/models/`, `app/schemas/`, `app/routers/`, `app/services/`, `app/main.py`;
- `README.md` com instalação, variáveis de ambiente e como rodar;
- `.env.example` com todas as variáveis (sem valores reais);
- `requirements.txt` OU `pyproject.toml` com versões fixadas.

5.2 Estrutura de Projeto (Frontend)

- `README.md` com instalação, variáveis de ambiente (`VITE_API_URL` etc.) e como rodar;
- Arquivo `.env.example` com as URLs dos módulos;
- Separação clara de camadas: serviços de API, componentes, páginas e rotas.

5.3 Padrões de API (Backend)

- Substantivos no plural: `/produtos`, não `/produto`;
- Status HTTP corretos: 200, 201, 204, 400, 401, 403, 404, 422;
- Paginação em listagens: `?page=1&size=10`;
- Erros no formato: `{"detail": "msg", "code": "ERRO"}`;
- Datas em ISO 8601 (UTC);
- CORS habilitado para a origem do Frontend.

5.4 Segurança

- Rotas protegidas (exceto `/health`, `/docs`, `/auth/login`) exigem JWT válido;
- Dados sensíveis fora de logs e respostas (senha, CPF completo);
- Frontend: token nunca em `localStorage` em ambiente de produção.

5.5 Testes (Backend)

- Mínimo **70%** de cobertura com `pytest-cov`;
- `TestClient` (`httpx`) — banco em memória nos testes;
- Ao menos 1 teste por operação: criar, listar, atualizar, deletar, regra de negócio.

5.6 Estrutura de Diretórios Sugerida

<code>modulo-financeiro/</code>	<code>frontend-portal/</code>
--- <code>app/</code>	--- <code>src/</code>
--- <code>core/</code>	--- <code>api/</code> # clientes HTTP

```
| |--- models/          | |--- components/
| |--- schemas/        | |--- pages/
| |--- routers/        | |--- store/      # estado global
| |--- services/       | |--- routes/
| \--- main.py         | |--- .env.example
|--- tests/            | |--- package.json
|--- alembic/          | \--- README.md
|--- requirements.txt
\--- README.md
```

6. Entregáveis

Todos os entregáveis devem ser submetidos via **pull request** no repositório do grupo com link no AVA. O prazo final e improrrogável é **01 de junho de 2025**.

#	Entregável	Responsável	Prazo
E1	Documento de Requisitos (RF + RNF) + Diagrama ER	Todos os grupos	12/05/2025
E2	Especificação OpenAPI exportada (openapi.json)	G0, G2, G3, G4, G5	16/05/2025
E3	Wireframes / protótipo de telas do Portal	G1 (Frontend)	16/05/2025
E4	Código-fonte versionado no GitHub (todas as branches)	Todos os grupos	26/05/2025
E5	Testes automatizados com cobertura $\geq 70\%$	G0, G2, G3, G4, G5	26/05/2025
E6	Documento de integração (contratos cross-módulo)	G2, G3, G4, G5	26/05/2025
E7	Frontend integrado a todos os módulos + dashboard	G1 (Frontend)	26/05/2025
E8	Script de seed com dados de exemplo	Todos os grupos	26/05/2025
E9	Apresentação final (10 min + demo ao vivo)	Todos os grupos	01/06/2025

Table 3: Entregáveis e prazos — prazo final: 01/06/2025

7. Cronograma Sugerido

Período	Datas	Atividade / Milestone
Semana 1	04–09/05	Sorteio dos grupos via sorteador.com.br • divisão de responsabilidades • setup do repositório GitHub • primeiro commit
Semana 2	10–16/05	Levantamento de requisitos (RF + RNF) • modelagem de dados (ER) • definição dos contratos de API • wireframes do Frontend
⇒ Entrega E1 (12/05) e E2 + E3 (16/05)		
Semana 3	17–23/05	Desenvolvimento da camada de dados e endpoints CRUD básicos (backend) • estrutura base do Frontend (rotas, autenticação, layout)
Semana 4	24–28/05	Regras de negócio • integração cross-módulo (Compras → Estoque / Financeiro) • dashboard do Frontend com dados reais • testes automatizados
⇒ Entrega E4, E5, E6, E7, E8 (26/05)		
Semana 5	29–30/05	Ajustes finais • correção de bugs • revisão da documentação • ensaio da apresentação
Apresentações	01/06	Demo ao vivo (10 min por grupo) + perguntas do professor

Table 4: Cronograma — prazo final 01 de junho de 2025

Atenção ao calendário: A semana 4 é curta (só 5 dias úteis até 28/05). Planejem-se para que os entregáveis E4–E8 estejam prontos até o dia 26/05, deixando 29–30/05 apenas para ajustes.

8. Critérios de Avaliação

A nota do grupo (0 a 10) é composta pelos critérios abaixo. Todos os integrantes recebem a mesma nota.

Critério	Peso (%)	Detalhamento
Levantamento de Requisitos (RF + RNF + ER)	20	Clareza, completude, rastreabilidade
Qualidade das APIs / Frontend (contratos, design)	20	Nomenclatura, status codes, UX básica
Integração entre módulos (incluindo Frontend)	20	Fluxos cross-module + dashboard real
Testes automatizados (backend)	15	Cobertura $\geq 70\%$
Documentação técnica (Swagger + README)	10	Auto-explicativo, exemplos
Apresentação / Demo ao vivo	15	Sistema rodando, clareza, segurança
TOTAL	100	<i>Nota máxima: 10,0</i>

Table 5: Critérios de avaliação

8.1 Penalizações

- $-1,0$ por entregável obrigatório ausente na data de apresentação;
- $-0,5$ se o sistema (backend ou frontend) não subir na hora da demo;
- $-0,5$ por endpoint documentado no Swagger que não funciona;
- $-1,0$ se o Frontend não estiver integrado a pelo menos 3 módulos;
- Plágio de código entre grupos $\Rightarrow$ nota **0** para ambos.

8.2 Bônus (até $+1,0$)

- $+0,5$ — Deploy em cloud (Railway, Render, Vercel, Fly.io. . .) com URL pública;
- $+0,5$ — Docker Compose orquestrando todos os serviços (backend + frontend).

9. Template de Documentação de Requisitos

9.1 Documento de Visão do Módulo

- Nome, responsabilidades e grupo responsável;
- Atores envolvidos (quem usa este módulo?);
- Requisitos Funcionais numerados (RF01...);
- Requisitos Não-Funcionais numerados (RNF01...);
- Restrições e premissas.

9.2 Diagrama de Entidades (Backend) / Wireframes (Frontend)

- **Backend:** Diagrama ER com chaves, cardinalidades e relacionamentos (dbdiagram.io, Draw.io ou Mermaid);
- **Frontend:** Wireframes das telas principais com fluxo de navegação (Figma, Excalidraw ou similar).

9.3 Especificação OpenAPI

- Exportar `openapi.json` da rota `/openapi.json` do FastAPI;
- Cada endpoint com `summary`, `description` e exemplos;
- Schemas Pydantic com `description` e `example` nos campos.

9.4 Documento de Integração (módulos G2–G5)

- Quais endpoints externos são chamados e em quais situações;
- Payload enviado e resposta esperada;
- Comportamento em caso de falha (timeout, 5xx) — rollback lógico.

9.5 Documento de Integração (Frontend — G1)

- Lista de endpoints consumidos por módulo com método e URL;
- Estratégia de gerenciamento de token JWT (armazenamento e renovação);
- Mapeamento tela → endpoint(s) utilizado(s).

10. Dicas e Referências

10.1 Boas Práticas

- **Backend:** comece pela modelagem de dados, depois os schemas Pydantic, depois as rotas;
- **Frontend:** implemente primeiro o login e a camada de serviços de API antes das telas;
- Use migrations (Alembic) desde o início — não altere o banco na mão;
- Commits pequenos e frequentes com mensagens no padrão Conventional Commits;
- Revisão de código via pull request — nenhum merge sem revisão de ao menos 1 colega.

10.2 Referências

- FastAPI: <https://fastapi.tiangolo.com>
- SQLAlchemy: <https://docs.sqlalchemy.org>
- Alembic: <https://alembic.sqlalchemy.org>
- JWT (python-jose): <https://python-jose.readthedocs.io>
- Testes FastAPI: <https://fastapi.tiangolo.com/tutorial/testing>
- Conventional Commits: <https://www.conventionalcommits.org/pt-br>
- Sorteio de grupos: <https://www.sorteador.com.br>

Bom trabalho a todos!

O aprendizado está no processo — não apenas no código final.

*Prazo final: **01 de junho de 2025.***